Project 03: Bayes Nets

Questions 1-4 and inference performance

Karan Jain - SBU ID: 117420953
Maitreya Vaghulade - SBU ID: 117437999

CSE 537 - Artificial Intelligence | October 29, 2026

1. Implementation and testing

We completed the four marked sections in `inference.py` and `factorOperations.py`. All 18 supplied support, layout and example-test files remain unchanged. Function names, signatures, constants, type checks, wrappers, operation tracking and original Berkeley attribution are preserved.

We tested the code with the four supplied examples and the additional tests included in the ZIP. An independent exhaustive oracle multiplies original CPT entries and sums complete assignments. It does not use joinFactors, eliminate or either inference routine. Section 6 lists the results and commands for running the tests.

The actual GUI was launched and arrow-key and WASD movement, sensor readings, and shading were visually verified.

2. Question 1: Bayes net structure

The network contains Pacman, Ghost0, Ghost1, Observation0 and Observation1. Observation0 has parents Pacman and Ghost0; Observation1 has parents Pacman and Ghost1. Each agent domain includes every (x, y) position in the grid, including walls. The maximum true Manhattan distance is (W - 1) + (H - 1). Adding MAX_NOISE = 7 gives an inclusive observation range from 0 through W + H + 5.

Case	Agent positions each	Observation values
Supplied 4 x 3 grid	12	0 through 12
Single-cell 1 x 1 grid	1	0 through 7
1 x 8; 9 x 1; 2 x 7; 8 x 5	8; 9; 14; 40	0-14; 0-15; 0-14; 0-18

Table 1. The supplied example and five additional grid shapes pass graph and domain checks. The function constructs an empty network; CPTs are assigned separately.

3. Question 2: Join factors

The result uses the union of input unconditioned variables. Its conditioned variables are the union of input conditioned variables minus all output unconditioned variables. A new factor preserves the complete domain dictionary. Each result entry is the product of matching input entries, without normalization; general factors need not be normalized probability tables.

Badge / Alarm	Expected product	Result
green / off	0.4 x 0.15 = 0.06	PASS
green / on	0.4 x 0.85 = 0.34	PASS
gold / off	0.6 x 0.8 = 0.48	PASS
gold / on	0.6 x 0.2 = 0.12	PASS

Table 2. Supplied Q2 results. Additional tests cover unnormalized values, zeros, general scopes, disconnected factors, iterator input and an independent copy for a single-factor join.

4. Question 3: Eliminate a variable

Elimination creates a new factor without the requested unconditioned variable. For each retained assignment, it sums the input entries over every value of the eliminated variable. Conditioned variables and the complete domain dictionary are preserved, and the input factor is unchanged. The supplied guards reject a missing or conditioned variable and elimination of the only unconditioned variable.

Zone / Badge	Expected marginal	Result
quiet / green	0.10 + 0.25 = 0.35	PASS
quiet / gold	0.40 + 0.25 = 0.65	PASS
busy / green	0.30 + 0.25 = 0.55	PASS
busy / gold	0.20 + 0.25 = 0.45	PASS

Table 3. Supplied Q3 results. Additional tests cover four-value and singleton domains, immutability, retained unused domains, exceptions and tracking.

5. Question 4: Variable elimination

CPTs are first specialized to the evidence. For each hidden variable in the requested order, the tracked join helper combines every factor containing that variable. Elimination follows immediately when the joined factor has more than one unconditioned variable; otherwise that factor is discarded as specified. The remaining factors are joined and normalized using the supplied helper.

If the elimination order is None, the supplied wrapper selects a sorted hidden-variable order. Normalization moves singleton-domain unconditioned variables, including evidence, to the conditioned set and returns None when the total mass is zero.

Fuel given Lamp=lit	Unnormalized joint mass	Posterior
full	0.222	111/202 = 0.549505
low	0.182	91/202 = 0.450495

Table 4. Supplied Fuel -> Sensor -> Lamp query, eliminating Sensor. The evidence probability is P(Lamp=lit) = 0.222 + 0.182 = 0.404. The two posterior entries sum to one.

6. Test results

Check	Result
Supplied Q1-Q4 examples	4/4 PASS
Additional factor/inference test methods	14/14 PASS
Randomized posterior queries (within the 14 methods)	1,155/1,155 PASS; 100 DAGs
Game integration test methods	2/2 PASS; 8 complete games

We extracted the completed ZIP to a fresh folder and ran both suites successfully. The random tests use seed 537, 2-6 binary or ternary variables, one or two query variables, up to two evidence variables, every hidden-variable permutation and the default order. Comparisons use relative tolerance 1e-10 and absolute tolerance 1e-12. The tests also check scopes, domains, unit mass, unchanged inputs and operation tracking.

Run from the extracted `bayesnets/` folder:

```
python3 test_q1to4.py
python3 -m unittest test_reference_solution test_game_demo -v
```

Eight games finished across all four layouts: four used stationary ghosts with scripted keyboard input and shading, and four used random ghosts with a test driver that could see their positions. These game checks are separate from Q1-Q4.

7. Performance comparison and critical analysis

We compared inference methods on binary star networks. Center is the parent of every Leaf, and the query is Leaf0 without evidence. Center-first eliminates Center before hidden leaves; leaves-first eliminates hidden leaves before Center. The values below are saved in `benchmark_results.json`. The benchmark used Python 3.12.5, seed 537 and PYTHONHASHSEED=1234.

Variables	Method	Time (ms)	Peak (KiB)
5	VE: center first	0.708	35.05
5	VE: leaves first	0.281	19.75
5	Enumeration	1.086	42.00
7	VE: center first	2.635	85.11
7	VE: leaves first	0.421	25.20
7	Enumeration	5.449	96.46
9	VE: center first	11.840	264.70
9	VE: leaves first	0.576	32.08
9	Enumeration	29.610	337.52

Table 5. Median runtime from 7 untraced queries; median peak Python allocation from 5 separate tracemalloc queries. Each method receives one warmup. Model construction and oracle checks are excluded; inference, evidence specialization and normalization are included. Each result is validated against the independent oracle.

Seven-variable method	Largest factor rows	Total created rows
VE: center first	128	382
VE: leaves first	4	30
Enumeration	128	1152

Table 6. The analyzer counts rows in newly created factor tables, including normalization. Running analyze_order.py reproduces these counts and the posterior.

Interpretation and effect of elimination order

For seven variables, leaves-first reduces the largest created factor from 128 rows to 4, a 32-fold difference, while preserving the posterior (0.3444444444, 0.6555555556). A hidden leaf has a normalized CPT and can be discarded after its tracked join. Eliminating Center first instead creates a factor coupling all leaves. At nine variables, leaves-first inference was about 51.4 times faster than enumeration.

Complexity and limitations

Variable elimination is exact up to floating-point rounding, but an intermediate table requires storage proportional to the product of its variable-domain sizes. Dense connectivity or an unfavorable order can therefore cause exponential growth. The supplied sorted default is deterministic rather than a performance heuristic. Choices such as min-fill can reduce intermediate factors, while evidence can shrink domains before joins.

These sparse synthetic models demonstrate order sensitivity, not performance on dense networks. Runtime depends on the machine, Python version and iteration order. Tracemalloc measures query-time Python allocation, not whole-process memory; factor-table rows are a distinct measure. Extremely small probabilities can underflow. The private grading suite was unavailable, so passing these checks does not guarantee full marks.

Run the benchmark with `PYTHONHASHSEED=1234 python3 benchmark.py`; reproduce table sizes with `PYTHONHASHSEED=1234 python3 analyze_order.py`. Timings can differ between runs. Based on the supplied CSE 537 handout and Berkeley Pacman framework (http://ai.berkeley.edu); original code notices are retained.